

다중 오라클 , 다중 탐색 그로버 알고리즘 아키텍처

조건 검색 (<, >, =, 범위) 을 만족하는 다중 타겟 Grover 가속기 — 스펙 결정
대상 FPGA : Spartan-7 XC7S100 | 보드 모델 확인 필요

PART 1

결정사항

무엇을 먼저 정해야 하는가

가속의 용도

	내용	판정
A. 시뮬레이터 가속	같은 양자 알고리즘을 PC 보다 빠르게 시뮬레이션한다. 비교 대상은 Qiskit AerSimulator / NumPy 상태벡터 시뮬레이터	채택
B. 알고리즘 프로토타이핑	실기 없이 조건 오라클이 수렴하는지 확인한다. 단 게이트 수준 검증은 불가 — 에뮬레이터의 오라클은 양자회로가 아니라 비교기 + 부호 반전	보조

고전 선형 스캔과는 비교하지 않는다

- 고전 검색은 $O(N)$ 이고 Grover 에뮬레이션은 $O(N \cdot \sqrt{N/M})$ 이라 , 이 가속기는 고전 스캔보다 느리다 .
- 가속 대상은 어디까지나 'PC 에서 돌리는 양자 시뮬레이터 ' 다 .
- 발표에서 반드시 나오는 질문이므로 입장을 먼저 고정한다 .

문제 정의

#	결정 항목	선택지	결정	연관된 자원
1	검색 대상	(a) 온칩 데이터 배열 value[i] (b) 인덱스의 함수 f(i)	(a)	BRAM
2	지원 조건문	< / > / = / 범위 a<x<b / 다중 키 AND·OR	4 종 (범위 포함)	비교기 개수 , 임계값 레지스터 폭
3	출력의 정의	(a) 조건 만족 원소 하나 (b) 전부 열거 (c) 개수 M 추정	(a) + 반복호출로 (b)	전부 열거하면 구조가 달라짐
4	정답 개수를 아는가	(a) 호스트가 사전 카운트 (b) BBHT 로 모른 채 탐색 (c) 양자 카운팅 (QPE)	(b) BBHT 기본 (a) 도 비교용으로 구현	(c) 는 복소수 강제 → 제외
5	검색 정확도	(a) ℓ2 노름 오차 (b) argmax 성공률 (c) top-M 회수율	(b) ≥ 99%	비트폭 결정
6	가속의 용도	A. 시뮬레이터 가속 B. 알고리즘 프로토타이핑	A	벤치마크 baseline

출력의 정의 ① — 다중 정답은 서로 구별되지 않는다

예시 : $N = 8$, 술어 $x = 12 \rightarrow$ 정답 $\{1, 3, 5\}$, 즉 $M = 3$

인덱스 i	0	1	2	3	4	5	6	7
$value[i]$	5	12	3	12	7	12	1	9
정답 ?		✓		✓		✓		

Grover 1 회 반복 후 ($R = 1$ 이 최적)

인덱스	0	1	2	3	4	5	6	7
진폭	-0.1768	0.5303	-0.1768	0.5303	-0.1768	0.5303	-0.1768	-0.1768
확률 a^2	1/32	9/32	1/32	9/32	1/32	9/32	1/32	1/32

- 세 정답의 진폭이 **0.5303** 으로 정확히 같다 . 오라클이 셋 다 똑같이 뒤집고 확산이 셋 다 똑같이 증폭하므로 대칭이 깨지지 않는다 .
- argmax 회로는 항상 인덱스 1 만 반환한다 . 몇 번을 다시 돌려도 1 이고 , 3 과 5 는 영원히 못 찾는다 .
- 출력을 '전부 열거' 로 정하면 측정 방식 (S3-5) 부터 바뀌어야 한다 .

출력의 정의 ② — 열거하는 두 가지 방법

	방식	평가
A. Born 확률 샘플링	진폭 크기에 비례해 무작위로 뽑고 중복을 버리며 모은다. 위 예에서 1, 3, 5 가 각각 9/32 확률	쿠폰 수집가 문제 — 중복 뽑기가 낭비
B. 마스킹	찾은 해를 오라클 조건에서 제외한다. if (조건 (i) && !found[i])	중복 없음 — 채택

방법 B 로 위 예시를 끝까지 돌리면

라운드	마킹되는 집합	M	R	성공률	결과
1	{1, 3, 5}	3	1	84.4%	1 발견 → found[1] = 1
2	{3, 5}	2	1	100%	3 발견 → found[3] = 1
3	{5}	1	2	94.5%	5 발견 → found[5] = 1
4	{ }	0	—	—	오라클이 아무것도 안 뒤집음 → 균등상태 유지 → 무작위 인덱스 → 고전 검증 실패 → 종료

· 4 라운드의 ' 더 이상 없다 ' 를 어떻게 판정할지가 새로 생기는 설계 문제다 .

출력의 정의 ③ — 하드웨어 비용은 작고 , 시간 비용은 크다

항목	하나만 반환	전부 열거 (방법 B)
측정 블록	argmax 비교기 트리 1 개	Born 샘플러 (LFSR + 누적확률 비교)
추가 메모리	없음	마스크 배열 N 비트 (n=16 → 64 Kb, BRAM 2 개)
오라클	비교기 1 개	비교기 + AND 게이트 1 개 (사실상 공짜)
FSM	S_DONE 에서 종료	S_DONE → 마스크 갱신 → S_INIT 복귀 외부 루프
종료 판정	불필요	'K 회 연속 검증 실패 시 종료 ' 규칙 필요
실행 시간	$\sqrt{N / M}$	$\sum \sqrt{N / k} \approx 2\sqrt{N \cdot M}$

n = 16 (N = 65,536), P = 32 기준 실행 시간

M	총 Grover 반복	사이클	@ 100 MHz	
1	201	823 K	8 ms	
100	5,100	21 M	210 ms	
1,000	16,200	66 M	0.66 s	
32,768 (x < T 로 절반이 걸린 경우)	92,700	380 M	3.8 s	고전 스캔이 5,000 배 빠름

- 열거 모드는 $M \ll N$ 일 때만 의미가 있다 . '=' 술어는 대개 M 이 작아 안전하지만 '<' / '>' 는 M 이 N/2 까지 갈 수 있다 .
- 스펙에 상한을 둔다 . $M > M_{\max}$ (예 : 256) 이면 열거를 포기하고 TOO_MANY 를 반환 .

정답 개수를 아는가 ① — BBHT

- 최적 반복 횟수는 $R = (\pi/4) \cdot \sqrt{N/M} - 1/2$ 이고 , 여기에 정답 개수 M 이 들어 있다 .
- 그런데 $value[i] < T$ 를 만족하는 칸이 몇 개인지는 데이터에 달려 있어 미리 알 수 없다 .
- M 을 모르면 회전각 θ 를 모르고 , θ 를 모르면 최적 반복 횟수를 계산할 수 없다 .

BBHT (Boyer - Brassard - Høyer - Tapp, 1998)

	단계
①	$m \leftarrow 1, \lambda \leftarrow 6/5$ 로 시작한다
②	$[0, m)$ 구간에서 정수 j 를 균등 무작위로 뽑는다
③	균등중첩에서 시작해 Grover 를 정확히 j 회 반복하고 측정한다
④	측정된 인덱스를 고전적으로 검증한다 . 조건을 만족하면 그것이 답이다
⑤	실패했으면 $m \leftarrow \min(\lambda \cdot m, \sqrt{N})$ 으로 상한을 키우고 ②로 돌아간다

- ④ 의 고전 검증이 에뮬레이터에서는 사실상 공짜다 — BRAM 에서 $value[i]$ 를 한 번 읽고 비교하면 끝 (1~2 사이클) .
- 실기에서는 측정이 상태를 붕괴시켜 회로를 통째로 재실행해야 하므로 이 검증이 비싸다 .
- m 을 1 부터 천천히 키우는 이유 — M 이 커서 한두 번이면 끝날 문제에 처음부터 j 를 크게 뽑으면 헛돈다 .

정답 개수를 아는가 ② — 왜 j 를 무작위로 뽑는가

성공 확률 $P(j) = \sin^2((2j+1)\theta)$ 는 j 에 대해 주기적으로 오르내린다 ($N = 8, M = 3, \theta = 37.76^\circ$)

j	$(2j+1)\theta$	성공 확률	
0	37.8°	37.5%	
1	113.3°	84.4%	최적
2	188.8°	2.3%	지나쳐서 폭락 — 골짜기
3	264.3°	99.0%	한 바퀴 더 돌아 다시 마루
4	339.8°	11.7%	

- 무작위화는 '좋은 j 를 잘 찾는 기법' 이 아니라 '나쁜 M 에 당하지 않게 하는 방어책' 이다.
- j 를 $[0, m)$ 에서 균등하게 뽑았을 때의 평균 성공 확률에는 닫힌 식이 있고, m 이 약 $\frac{1}{2}\sqrt{(N/M)}$ 을 넘으면 그 값이 θ 와 무관하게 25% 아래로 내려가지 않는다.
- 즉 M 을 몰라도 한 샷 성공률의 하한이 보장되고, 기대 4 샷 안에 끝난다.
- 반대로 j 를 m 의 함수로 못 박으면, 그 규칙이 만드는 j 들이 하필 골짜기에 연달아 떨어지게 하는 M 이 언제나 존재한다.
- 최적은 아니다 — M 을 알 때 $0.79 \cdot \sqrt{(N/M)}$ 대 BBHT $4.5 \cdot \sqrt{(N/M)}$, 약 5.7 배 손해. 다만 차수는 최적 (BBBV 하한, 1997).
- 결정 : 두 모드를 모두 구현하고 기본값을 BBHT 로 한다. 5.7 배가 실측되는지 확인하면 그 자체가 결과가 된다.

검색 정확도 — 오차는 오라클이 아니라 산술에서 생긴다

연산	하는 일	오차
오라클 판정	$\text{value}[i] < T \rightarrow$ 정수 비교, 뺄셈 후 MSB	없음 — 완전 정확
부호 반전	$\text{amp}[i] \leftarrow -\text{amp}[i] \rightarrow$ 2의 보수 부정	없음
합산	$\sum a_i \rightarrow$ 정수 덧셈 (누산기가 충분히 넓으면)	없음
평균	$\bar{a} = (\sum a_i) \gg n$	여기서 발생
확산	$a_i \leftarrow 2\bar{a} - a_i$	위 오차가 전파
초기화	$a_i \leftarrow 1/\sqrt{N}$	여기서 발생

- '크면 뒤집는다'는 비트 연산이라 오차가 없다. 오차는 실수 산술을 하는 곳에서만 생긴다.
- 특히 $1/\sqrt{N}$ 은 무리수라 유한 비트로 정확히 답을 수 없다. $n=16$ 이면 $1/\sqrt{N} = 1/256 = 0.0039$ 인데, 소수부가 4비트면 이 값이 0으로 반올림되어 알고리즘이 통째로 죽는다.
- 소수부 f 는 최소 $n/2$ 를 넘어야 하고, 여기에 R 회 반복의 누적 오차만큼 여유가 더 필요하다.
- 고장 모드 셋 — ① 진폭 순위 뒤집힘 (argmax 오답) ② 회전각 드리프트 ③ 진폭 소멸
- 다행히 우리는 진폭의 절대 정확도가 아니라 순위만 보존하면 된다. 이 느슨함이 비트폭을 크게 아껴 준다.

양자화 / 자릿수

#	결정 항목	선택지	결정
S1-1	실수 vs 복소수	실수 전용 / 복소수	실수 전용 — 양자 카운팅을 뺐으므로 위상은 0 또는 π 뿐
S1-2	수 포맷	고정소수점 / float32 / block FP	고정소수점
S1-3	진폭 Q 포맷	정수부 i + 소수부 f	Q1.17 (18 비트) — 골든 모델 실측 후 확정
S1-4	누산기 폭	f + n 이상	f + n + 2
S1-5	반올림	절단 / 최근접 반올림 / 확률적	최근접 반올림 — 절단은 편향이 R 회 누적됨
S1-6	오버플로	포화 / 랩	포화 — 랩은 부호가 뒤집혀 결과가 완전히 깨짐
S1-7	재정규화	없음 / 라운드마다 / block FP	없음 — Grover 는 노름 보존이 이론상 보장
S1-8	데이터 워드 폭 W	8 / 16 / 32 비트, 부호 유무	signed 16 비트

문제 복잡도 / 메모리

#	결정 항목	결정	근거
S2-1	목표 큐비트 n	16 (설계 목표), 12 에서 시작	BRAM 예산표 (다음 장)
S2-2	저장 위치	BRAM 온칩 전용	빠르고 간단 . 외부 메모리 주변회로 불필요
S2-3	데이터 배열 온칩 여부	온칩	스트리밍하면 매 반복마다 재로드 → 대역폭 병목
S2-4	병렬도 P	32 레인	성능을 지배하는 유일한 파라미터
S2-5	뱅킹 구조	인덱스 하위 $\log_2 P$ 비트로 뱅크 선택	순차 스트리밍이라 뱅크 충돌 없음

· $n = 16$, 진폭 18 비트면 진폭 배열이 정확히 BRAM36 32 개를 차지한다 . $P = 32$ 뱅킹이 하드웨어적으로 자연스럽다 .

아키텍처

#	결정 항목	선택지	선택
S3-1	시스템 형태	(a) 순수 RTL + UART (b) MicroBlaze + AXI (c) standalone FSM	(b) — BBHT 외부 루프를 SW 로 두면 훨씬 쉬움 ※ 우리는 RVX 라 펌웨어 올리는 방법 확인 필요
S3-2	역할 분담	진폭 갱신 = HW / BBHT 루프 · T 갱신 · 검증 = SW	위와 동일
S3-3	패스 구조	1 패스 융합 / 2 패스 (오라클 + 합 → 확산)	2 패스 — 평균은 전체 합이 끝나야 나옴
S3-4	R 계산	실시간 제공근 / ROM LUT / 상수 시프트	ROM LUT — m 값 종류가 30 개 남짓
S3-5	측정	argmax / Born 확률 샘플링 (LFSR + CDF)	Born 샘플링 — 다중 정답의 진폭이 같아 argmax 로는 열거 불가
S3-6	곱셈기	DSP 사용 / shift+sub only	확산은 shift+sub only (DSP 0 개) . 측정 제공기에만 DSP 1 개
S3-7	목표 클럭	100 / 150 / 200 MHz	우선 100 MHz

검증 / 평가 방식

#	결정 항목	내용
S4-1	골든 모델	Python/NumPy float64 → 비트정확 고정소수점 모델 → RTL, 3 단 비교 ※ 소프트웨어로 먼저 구현해 보는 것으로 !
S4-2	합격 기준	골든 모델 대비 bit exact (RTL vs 고정소수점 모델)
S4-3	벤치마크 축	$n \in \{8, 10, 12, 14, 16\}$ × $M \in \{1, 2, 4, \dots, N/2\}$ × 술어 4 종 × 데이터 분포 3 종
S4-4	비교 baseline	Qiskit AerSimulator (statevector) + NumPy 직접 구현, 둘 다
S4-5	평가 지표	성공률 / 반복 횟수 / 사이클 / LUT·FF·BRAM·DSP 사용률 / 전력

· Phase 1(골든 모델) 을 RTL 보다 먼저 하는 이유 — 비트폭 f 를 논문 공식으로 베끼면 32 비트가 나오지만 실측하면 18 비트로 충분할 가능성이 크다. 이 차이가 n 을 16 까지 올릴 수 있느냐를 가른다.

예상되는 자원 사용량 — 추정치

- XC7S100 기준 : BRAM $120 \times 36 \text{ Kb} = 4,320 \text{ Kb}$ (약 552 KB) | DSP 160 개 | 로직 셀 102,400 개
- 정확한 보드 모델은 미확인 — 보드를 가지고 있는 분에게 확인할 것
- 아래는 진폭 18 비트 + 데이터 16 비트 기준

n	N	진폭 배열	데이터 배열	합계	BRAM 점유	판정
12	4,096	72 Kb	64 Kb	136 Kb	3%	여유
14	16,384	288 Kb	256 Kb	544 Kb	13%	여유
16	65,536	1,152 Kb	1,024 Kb	2,176 Kb	50%	목표
17	131,072	2,304 Kb	2,048 Kb	4,352 Kb	101%	불가
18	262,144	4,608 Kb	4,096 Kb	8,704 Kb	202%	불가

- $n = 16$ 이 온칩 천장이다 . 절반을 남기는 이유는 마스크 배열 , BBHT 상태 , 파이프라인 버퍼 , 라우팅 여유 .
- 보드에 DDR3 가 달려 있다면 S2-2 를 다시 열어 n 을 20 이상으로 키울 수 있다 (대신 순차 스트리밍이 강제됨) .

비트폭

05 번 논문의 공식 $f_{\min} = \lceil n - \log_2(\ell_2 \text{ 허용오차}) - 1.03 \rceil$ 을 $n = 16$ 에 적용하면

기준 (S0-5)	요구	$f_{\min}$	진폭 워드
ℓ_2 오차 10^{-5}	엄격	32 비트	33 비트
ℓ_2 오차 10^{-3}	보통	25 비트	26 비트
argmax 성공률 99%	순위만 보존	약 17 비트 (추정)	18 비트

- 정확도 기준을 **argmax** 성공률로 잡는 것이 이 설계 최대의 자원 절감 레버다 .
- 우리는 진폭의 절대 정확도가 아니라 순위만 보존하면 되기 때문이다 .
- 다만 위 17 비트는 계산이 아니라 추정이다 . 공식이 ℓ_2 기준이라 argmax 기준으로 그대로 옮길 수 없다 .
- 골든 모델에서 f 를 8~32 로 스위치 성공률 곡선을 실측한 뒤 확정한다 .
- 18 비트를 잠정 채택하는 실용적 이유 : BRAM36 은 2048×18 구성이 네이티브라 비트 낭비가 0 이다 .

병렬도

$n = 16, M = 1$ 기준 $R = (\pi/4) \cdot \sqrt{65,536} \approx 201$ 회. 반복당 2 패스 $\times N$ 원소 = 131,072 원소 접근

P	반복당 사이클	총 사이클	@ 100 MHz	NumPy(추정 ~80 ms) 대비
1	131,072	26.3 M	263 ms	0.3× (느림)
8	16,384	3.3 M	33 ms	2.4×
32	4,096	823 K	8.2 ms	약 10×
64	2,048	412 K	4.1 ms	약 20×

- $P = 1$ 로 만들면 PC 보다 느리다. 성능을 정하는 것은 비트폭이 아니라 병렬도다.
- 비트폭을 30 비트에서 18 비트로 줄이면 메모리를 아끼지만 성능은 그대로인 반면, P 는 성능을 직접 32 배 바꾼다.
- $n = 16 \cdot 18$ 비트에서 진폭 배열이 정확히 BRAM36 32 개이므로 $P = 32$ बैंकिंग이 자연스럽다.
- Qiskit 대비로는 이보다 훨씬 큰 배수가 나올 것으로 예상된다.

PART 2

양자컴퓨터와 에뮬레이터 동작 방식 비교

$N = 16$, 상자 16 개에서 42 를 찾는 예제로 전 과정을 따라간다

양자컴퓨터 ① — 문제 설정과 반복 횟수

상자 16 개 (큐비트 4 개) 에 숫자가 들어 있고 , 42 가 든 상자를 찾는다

인덱스	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
값	7	19	3	25	11	8	30	14	2	22	42	5	17	9	28	13

- 정답은 인덱스 10 (이진수 1010), $M = 1$.
- 양자컴퓨터는 이것을 모른다 . 아는 것은 판정 규칙 $\text{value}[i] == 42$ 뿐이다 .

반복 횟수 계산 (기계가 시작 전에 정한다)

$$\sin \theta = \sqrt{(M/N)} = \sqrt{(1/16)} = 1/4 \quad \rightarrow \quad \theta = 14.478^\circ$$

$$R = \pi/(4\theta) - 1/2 = 3.108 - 0.5 = 2.61 \quad \rightarrow \quad R = 3$$

0 단계 — 큐비트 준비

- 큐비트 4 개를 $|0000\rangle$ 으로 놓고 아다마르 게이트 4 개를 건다 . 게이트 딱 4 개로 16 개 진폭이 전부 $1/4$ 이 된다 .
- 이 시점의 정답 확률 6.25% 는 눈 감고 찍는 것과 똑같다 . 여기서 시작한다 .

양자컴퓨터 ② — 증폭 과정

단계	정답칸 (10 번) 진폭	나머지 15 칸 진폭	평균 $\bar{a}$	정답 확률
초기 (H 게이트)	0.25000	0.25000	0.25000	6.25%
반복 1 회	0.68750	0.18750	0.21875	47.3%
반복 2 회	0.95313	0.07813	0.13281	90.8%
반복 3 회 ← 최적	0.98047	-0.05078	0.01367	96.1%
(반복 4 회 — 지나침)	0.76270	-0.16699	-0.10889	58.2%

- 오라클은 크기를 바꾸지 않고 부호만 뒤집는다 . 오라클 직후에 측정하면 여전히 6.25% 씩이다 — 부호는 측정에 보이지 않는다 .
- 확산 $a \leftarrow 2\bar{a} - a$ 가 그 부호 차이를 크기 차이로 번역한다 . 정답이 음수라 평균이 내려가고 , 그 내려간 평균이 지렛대가 된다 .
- 3 회에서 평균이 거의 0 까지 내려온다 . 정답 하나가 나머지 15 개를 합친 것과 맞먹기 때문이다 .
- **4 회에서 평균이 음수로 넘어가면 확산이 정답을 깎기 시작한다 . 96.1% → 58.2%. 회전이므로 지나치면 되돌아온다 .**

양자컴퓨터 ③ — 측정과 고전 검증

반복 3 회 후 최종 상태

인덱스	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
진폭	−.051	−.051	−.051	−.051	−.051	−.051	−.051	−.051	−.051	−.051	.980	−.051	−.051	−.051	−.051	−.051
확률	.26%	.26%	.26%	.26%	.26%	.26%	.26%	.26%	.26%	.26%	96.1%	.26%	.26%	.26%	.26%	.26%

측정 — 딱 한 번

- 측정 버튼을 누르면 기계가 위 확률표에 따라 숫자 하나를 뱉는다. 96.1%로 10, 3.9%로 나머지 15개 중 하나.
- 이 순간 중첩은 사라지고 상태는 $|1010\rangle$ 하나만 남는다.
- 기계가 알려준 것은 정수 10 하나뿐이다. 위 진폭 표는 우리가 계산한 것이지 기계에서 읽어낸 것이 아니다.

고전 검증

- `value[10] == 42 ?` → `42 == 42` → 참. 답은 인덱스 10.
- 확률이 아니라 검산으로 확정된 사실이다. 틀린 답을 맞다고 믿는 경우는 생기지 않는다.
- 만약 7이 나왔다면 `value[7] = 14 ≠ 42`로 실패하고 회로를 처음부터 다시 돌린다. 2 연속 실패 확률은 0.15%.

에뮬레이터 — 무엇이 같고 무엇이 다른가

양자컴퓨터와 달리 우리는 '측정 전까지 중간 확률을 알 수 없다'도 '측정 시 확률이 붕괴되어 다시 계산해야 한다'도 성립하지 않는다. 그럼에도 방식을 동일하게 모방하기 위해 중간 값을 검사하지 않고, 측정마다 진폭을 초기화한다.

항목	실제 양자컴퓨터	우리 에뮬레이터
진폭 전체 관찰	불가 — 측정하면 붕괴	가능하지만 일부러 하지 않음
측정	Born 확률로 인덱스 1 개	LFSR 난수 + 누적확률 비교로 동일하게 흉내
실패 후 재시도	회로를 처음부터 재실행	진폭 배열 전체를 INIT_AMP 로 재기입 (S_INIT 복귀)
반복 1 회 비용	$O(n)$ 게이트 — 수십 개	$O(N)$ 진폭 갱신 — 65,536 개
부호 반전의 정체	마이크로파 위상 180° (virtual Z, 시간 0)	2 의 보수 부정 — $\text{amp}[i] \leftarrow -\text{amp}[i]$
판정 · 루프 · 검증	고전 제어 컴퓨터	MicroBlaze 또는 호스트 PC

- 반복 1 회 비용의 $O(n)$ 대 $O(N)$ 격차가 에뮬레이터가 치르는 대가다. 실기는 큐비트 16 개를 건드리면 진폭 65,536 개가 물리적으로 동시에 바뀌지만, 우리는 하나씩 꺼내 계산해야 한다.
- 치트 모드 (오라클 후 음수 진폭을 스캔하면 정답 전부를 N 사이클에 얻음) 는 골든 모델 대조용 디버그 기능으로만 두고 동작 경로에는 넣지 않는다.